Summary

The first approach we took was to build an arbitrage trader, and then to try and attack bugs in our classmate's exchanges. To do this safely, we built three tools with the help of Claude: a read-only market scanner, an atomic arbitrage contract, and an EVM bytecode analyzer.

Our findings are that the marker is very efficient and all exchanges were well-built. Every pool trades each token at the same price, and because AMMs don't charge a fee, the price doesn't leave any room for arbitrage. We checked the bytecode of every exchange that held a non-negligable amount of FakeEth, and found that all of them were secure against vulnerabilities.

Our market differs from a real world market in many ways. The total FakeEth and tokens are fixed, meaning any trading is zero-sum, or slightly negative with gas fees; There's also no inflow and no issuance of new tokens.

Our conclusion is that since all markets and contracts can be well written with AI, and since our market is stagnant with no outside influence to change prices, there is no profitable attack or arbitrage available. The best way to 'win' this game is to hold on to our FakeEth and not pay any gas fees.

Tools Build

scan_exchange.js: a read-only script that runs against lu-eth. For every exchange address, it reads the Eth and ERC-20 reserves, computes the implied price, calls each estimate* view, and reports the price spread per token. It doesn't send transactions, and just lets us better understand the market.

ArbBot.sol: an atomic two-leg arbitrage contract. It takes a buy exchange and a sell exchange, swaps them in a single transaction, and reverts unless it ends with a defined minimum profit in FakeETH. By making the contract atomic, we eliminate the risk of a third party trading between our two legs. The worst case is a reverted transaction that costs us gas.

analyzer_env.py: a EVM disassembler that maps a pool's bytecode to the H4 interface functions and flags any reentrancy bugs.

Market Reconnaissance

Using the block explorer's token-holder API, we looked over every contract holding ASA, HAW, or KOR, a total of 22 exchange addresses plus our own three exchanges.

We found that the market is at nearly perfect price parity. Within each token, every pool quotes virtually the same price.

Token	Price (FakeEth per token)	# live pools
ASA	0.00516198	6
HAW	0.00674791	8
KOR	0.00635024	7

Because the AMMs don't charge a swap fee, once prices are equal across pools there's no arbitrage to capture. Any round trip would return exactly what we put in. There's also no triangular arbitrage, because the only trading pairs are token/ETH (no ASA<->HAW markets).

We did find one address that showed HAW prices ~97% below market value, but after examining the bytecode we discovered it wasn't actually an exchange. It was a leftover trading bot made by another classmate, holding 3.36 HAW and 0.0006 FakeEth as leftover inventory.

Our only Arbitrage Attempt

Before we understood that the previously mentioned bot was a bot, we tried to make a small trade to buy its cheap HAW and sell it on another pool. The transaction reverted inside the swap call and only cost us gas. While this was slightly disappointing, it showed that our ArbBot contract was well designed and that the principal token was never at risk. If our transaction wasn't atomic within our contract, we could have accidentally sent FakeEth and got nothing in return.

Security Audit of the Exchanges

Since no arbitrage existed, the only other way to make FakeEth was to exploit a buggy exchange. We disassembled the deployed bytecode of every contract that holds meaningful FakeEth.

Implementation	Pools	FakeEth	Reentrancy defense	Verdict
A (0x993b5a)	3 (ASA/HAW/KOR)	~2.3	ReentrancyGuard	secure
B (0x7dae7b)	3 (ASA/HAW/KOR)	~2.34	none but CEI-correct	secure
C (0xe594d7 *)	3 (ASA/HAW/KOR)	~0.97	boolean guard	secure
D (0x76168d)	3 (ASA/HAW/KOR)	~0.24	ReentrancyGuard	secure

* Implementation C compiles the same source three times with a different immutable token address, so its three pools have different bytecode hashes but are one codebase.

We checked each for four of the most likely vulnerabilities to exist:

Reentrancy (DAO bug): To make a profit from this bug, we want the contract to re-enter withdrawLiquidity during the ETH payout but before the contract decrements our position. Implementations A, C, and D have an explicit reentrancy guard. Implementation B has no guard, but it's still safe because it writes liquidityPositions, totalLiquidityPositions, and K to storage before transferring tokens or sending ETH, so a re-entrant call sees the already-updated state and gains nothing. In all our classmate's implementations, the ETH transfer is the last action.

msg.value double-counting: In the payable swapForERC20Token, the sent ETH is already in address(this).balance, so the correct post-swap balance is SELFBALANCE, not SELFBALANCE + msg.value. All four implementations compute tokenOut = tokenBalance - K / SELFBALANCE, which is correct. None double count, so there's no way to get cheap tokens.

Missing / loose ratio check in provideLiquidity: If a pool didn't enforce that deposited ETH and tokens match the current reserve ratio, you could deposit lopsidedly and withdraw more than you put in. All four enforce the ratio: B with a +-1 unit rounding tolerance, C and D with exact cross-multiplied equality.

Pool-draining via withdrawLiquidity: All four reject burning the entire pool by rejecting with an error message when we try to withdraw more than totalLiquidityPositions. Implementation D additionally bounds every swap output to 0 < out < reserve and recomputes K locally each call rather than trusting a cached value.

The only pools we didn't audit are three tiny or empty pools with negligible values not worth the gas fees to try and attack.

Why everything was secure

Despite our efforts to seek out bugs, the market was airtight. Everyone had access to AI assistants, so the common pitfalls a developer might make were all caught before deployment.

Holdings

	FakeEth	ASA	HAW	KOR
Before	9.988	0	0	0
After	9.978 (gas fees)	0	0	0

Files Included

CSE-242-h6-final-report.pdf:	this document
H6_market_log.md:	the chronological working log of everything we did.
ArbBot.sol:	the atomic arbitrage contract.
scan_exchanges.js:	the read-only market scanner
analyzer_evm.py:	the EVM bytecode analyzer used for the security audit.
scan_output.json:	the live market snapshot

Important Transactions we Made

Action	**Address / tx**	**Result**
Deploy ArbBot	0xAde3f9f0C4B4aDF9aDb7057b4f5ACeebA7724710	success
Fund ArbBot 0.01	tx 0xc795...95c6	success
Arbitrage attempt vs 0xc795...95c6	tx 0x4eec...0628	reverted (by design, no loss)